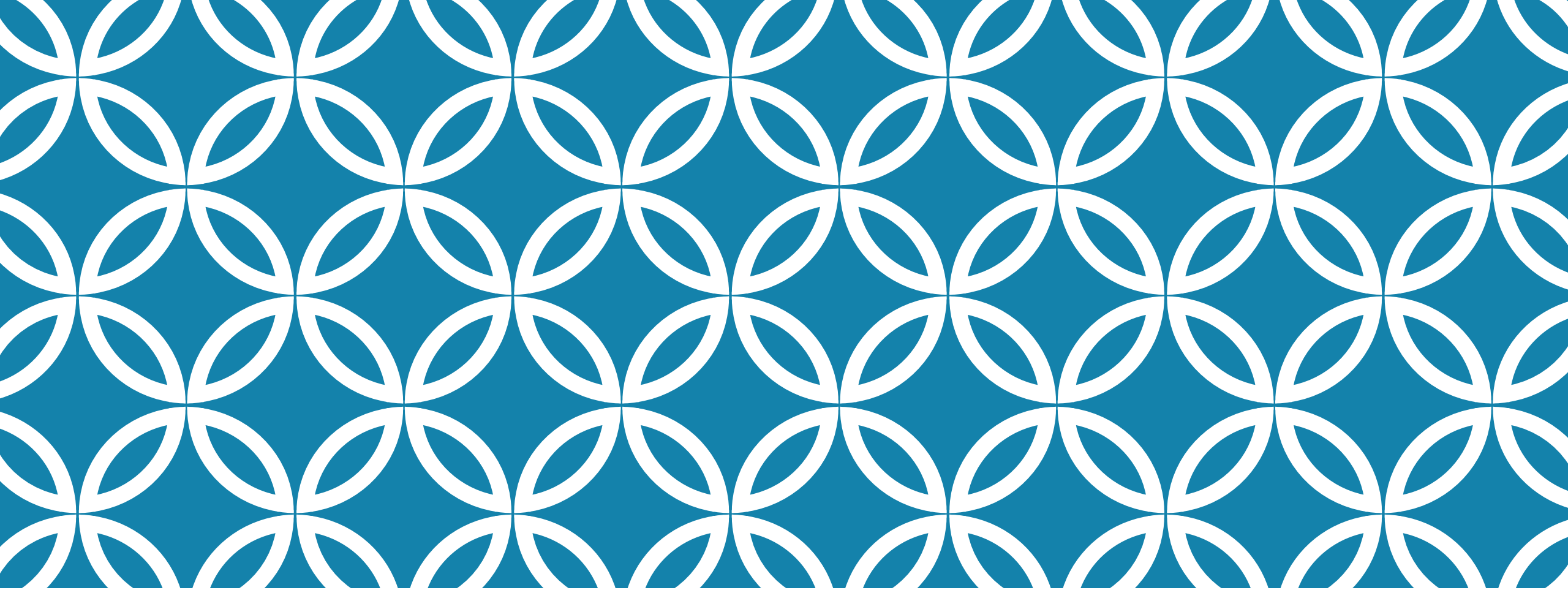


Inheritance in OOP

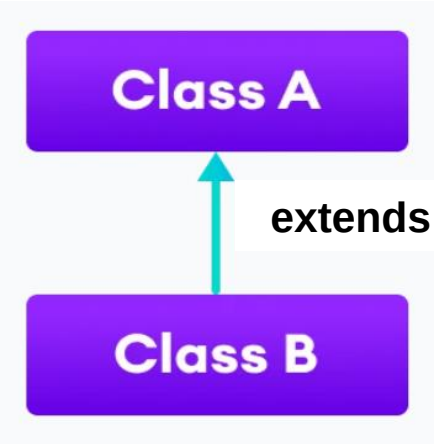
- **Inheritance**
- Using keywords **super** and **final**

OUTLINE

- Introduction to inheritance
- Types of inheritances
- Special keywords: **super** and **final**
- Examples

Introduction

(**New class B**)
- Child class or
- Sub class or
- Derived class



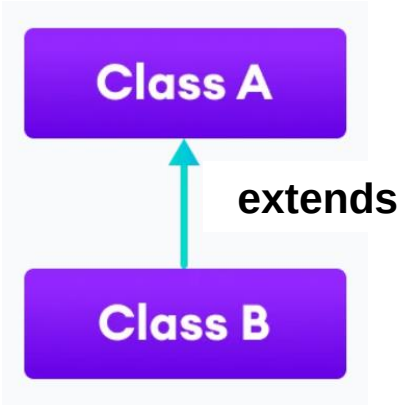
(**Existing class** A)
- Parent class or
- Super class or
- Base class

- **Inheritance** is one of the core concepts of Object-Oriented Programming (OOP) in Java.

- It allows a class (**subclass/child class**) to inherit the data properties and methods of another class (**superclass/parent class**).
- This promotes **code reusability** and establishes a **relationship between classes.**

INHERITANCE

(**New class B**)
- **Child** class
- Sub class
- Derived class



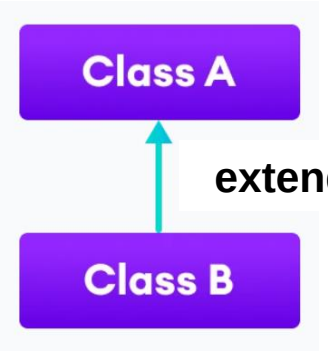
(**Existing class** A)
- **Parent** class
- Super class
- Base class

- **Inheritance** is one of the key features of OOP that allows us to create a new class from an existing class.
  - All **data** and **methods** that are not private in the existing class **can be used in the new class.**

- The new class is called **subclass** (**child class** or **derived class**) and the existing class is called **superclass** (**parent class** or **base class**).
  - All **data** and **methods** in parent class **can be used in the child class, if they are not private.**

INHERITANCE

(New class B)
- Child class or
- Sub class or
- Derived class



(Existing class A)
- Parent class or
- Super class or
- Base class

- The **extends** keyword is used to perform inheritance in Java

```
class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;
 public static void main(String args[]){
   Programmer p=new Programmer();
   System.out.println("Programmer salary is:"+p.salary);
   System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

```
class Maths {
    int addition(int a, int b) {
        return a + b;
    }
    int subtraction(int a, int b) {
        return a - b;
    }
}

public class Calculation extends Maths{
    int multiplication(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        Calculation calculation = new Calculation();
        System.out.println("Addition : "+ calculation.addition(20, 10));
        System.out.println("Subtraction : "+ calculation.subtraction(20, 10));
        System.out.println("Multiply : "+ calculation.multiplication(20, 10));
    }
}
```

Inheritance

Class Diagram

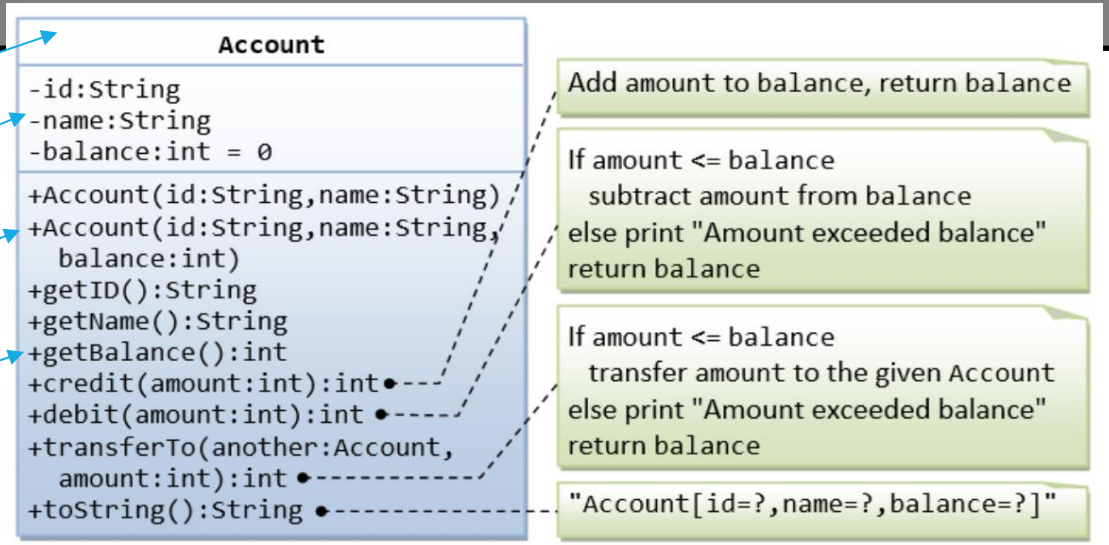
- A **class diagram** is a type of UML (Unified Modeling Language) diagram used to visually represent the structure of a software system.
- It describes the blueprint of a system by showing:
  Classes
  Attributes (fields)
  Methods (operations)
  Relationships between classes (such as inheritance, associations, and dependencies)

Class name

Attributes

Constructors

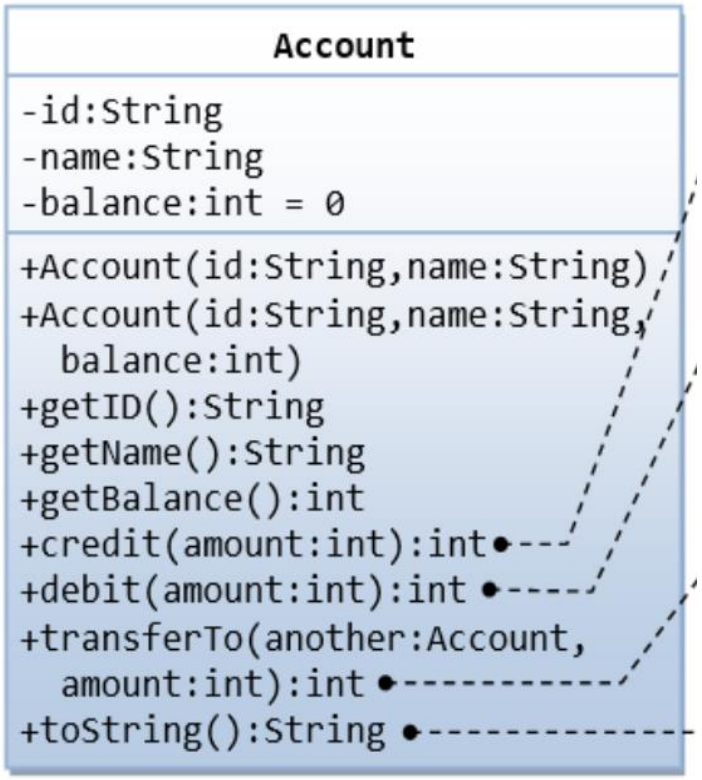
Methods



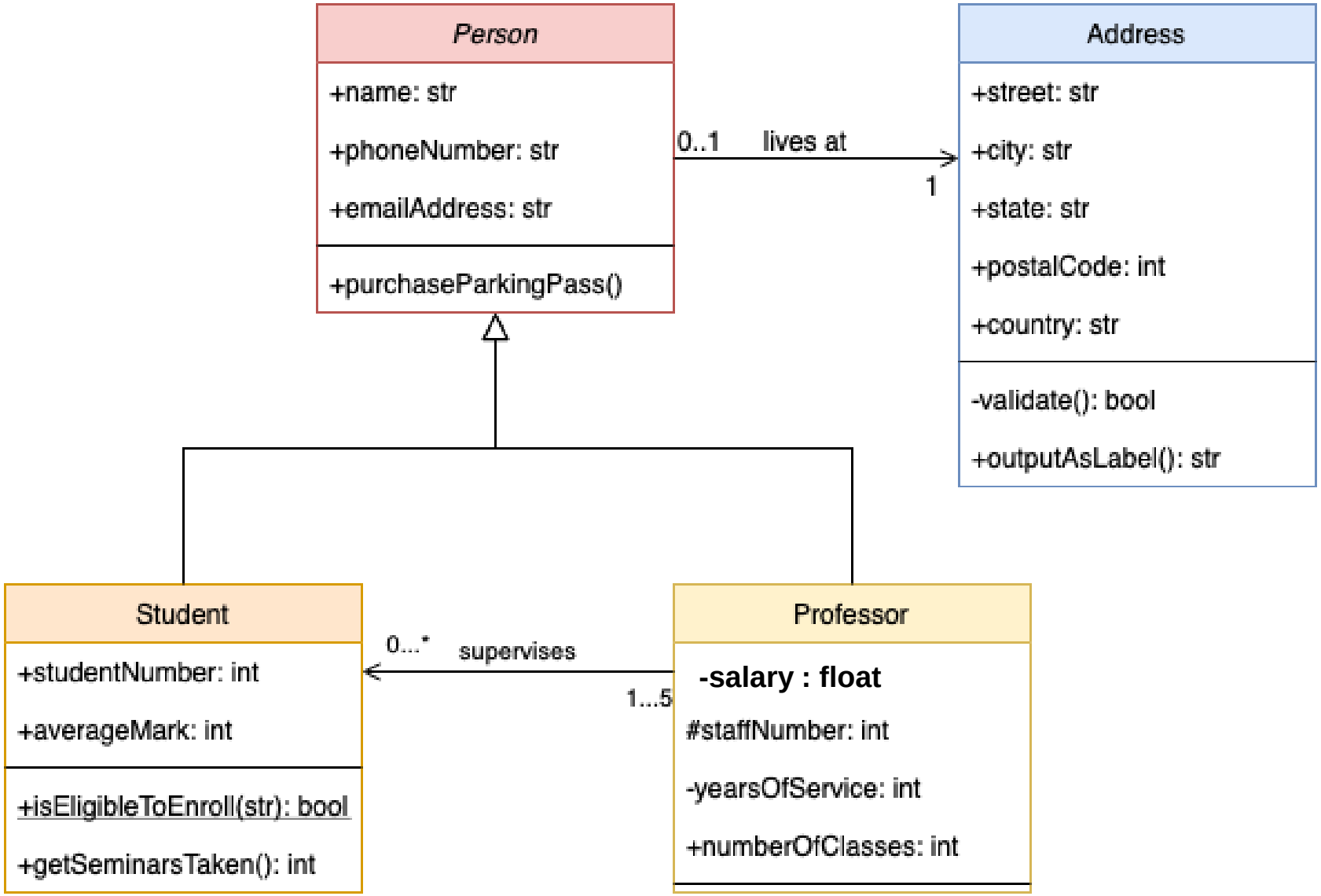
- \- : private
- \+ : public
- \# : protected
- ~ : package access

Account.java

```java
package classdiagram;
public class Account {
    private String id;
    private String name;
    private int balance = 0;

    Account(String id, String name){
        this.id = id;
        this.name = name;
    }
    Account(String id, String name, int balance){
        this.id = id;
        this.name = name;
        this.balance = balance;
    }

    public String getID() {
        return id;
    }
    public String getName() {
        return name;
    }
    public int getBalance() {
        return balance;
    }
    public int credit(int amount) {
        balance = balance + amount;
        return balance;
    }
}   ……
```



-	: private
+	: public
#	: protected
~	: package access

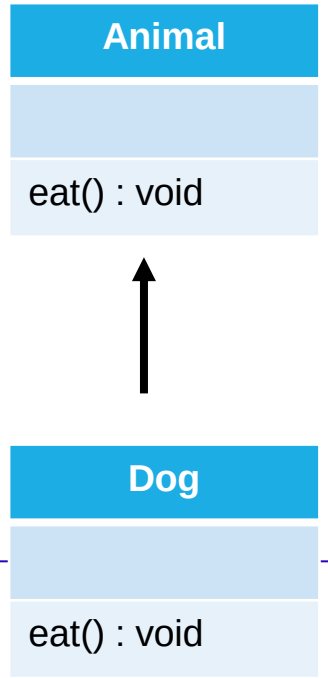

Person
+name: str
+phoneNumber: str
+emailAddress: str
+purchaseParkingPass()
0..1
lives at
1
Address
+street: str
+city: str
+state: str
+postalCode: int
+country: str
-validate(): bool
+outputAsLabel(): str
Student
+studentNumber: int
+averageMark: int
+isEligibleToEnroll(str): bool
+getSeminarsTaken(): int
0...*
supervises
1...5
Professor
-salary : float
#staffNumber: int
-yearsOfService: int
+numberOfClasses: int

INHERITANCE

- In Java, inheritance is an **is-a relationship**. That is, we use inheritance only if there exists an is-a relationship between two classes.
- For example:
  - Car is a Vehicle
  - Orange is a Fruit
  - Surgeon is a Doctor
  - Dog is an Animal
- Here, Car can inherit from Vehicle, Orange can inherit from Fruit, and so on.

INHERITANCE

- **Overriding methods**
  - When child class have the same methods to those methods in the parent class.




```java
class Animal {

  // method in the superclass
  public void eat() {
    System.out.println("I can eat");
  }
}

// Dog inherits Animal
class Dog extends Animal {


  public void eat() {
    System.out.println("I eat dog food");
  }

  // new method in subclass
  public void bark() {
    System.out.println("I can bark");
  }
}

class Main {
  public static void main(String[] args) {

    // create an object of the subclass
    Dog labrador = new Dog();

    // call the eat() method
    labrador.eat();
    labrador.bark();
  }
}
```

INHERITANCE

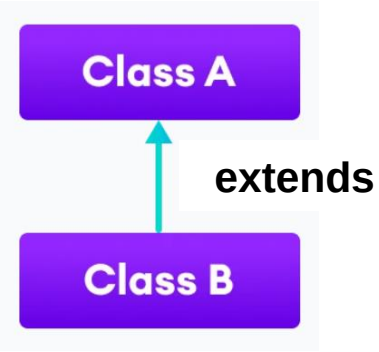
- The most important use of inheritance in Java is **code reusability**. The codes (data, methods, and constructors) in the parent class can be directly used by the child class.

- **Method overriding** is also known as *runtime polymorphism*. Hence, we can achieve Polymorphism in Java with the help of inheritance.

```java
class Animal {
  protected String name;

  protected void display() {
    System.out.println("I am an animal.");
  }
}

class Dog extends Animal {

  public void getInfo() {
    System.out.println("My name is " + name);
  }
}

class Main {
  public static void main(String[] args) {

    // create an object of the subclass
    Dog labrador = new Dog();

    // access protected field and method
    // using the object of subclass
    labrador.name = "Rocky";
    labrador.display();

    labrador.getInfo();
  }
}
```

INHERITANCE

(New class B)
- Child class or
- Sub class or
- Derived class



(Existing class A)
- Parent class or
- Super class or
- Base class

- The extends keyword is used to perform inheritance in Java

```
class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;
 public static void main(String args[]){
   Programmer p=new Programmer();
   System.out.println("Programmer salary is:"+p.salary);
   System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

```
class Maths {
    int addition(int a, int b) {
        return a + b;
    }
    int subtraction(int a, int b) {
        return a - b;
    }
}

public class Calculation extends Maths{
    int multiplication(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        Calculation calculation = new Calculation();
        System.out.println("Addition : "+ calculation.addition(20, 10));
        System.out.println("Subtraction : "+ calculation.subtraction(20, 10));
        System.out.println("Multiply : "+ calculation.multiplication(20, 10));
    }
}
```

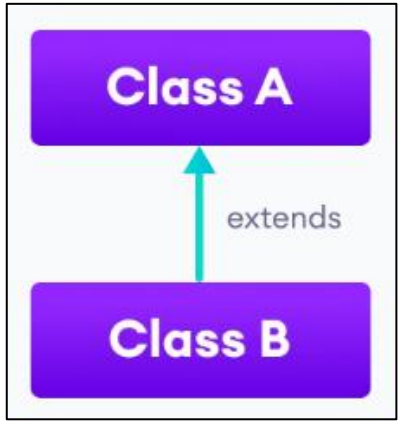
Inheritance

INHERITANCE

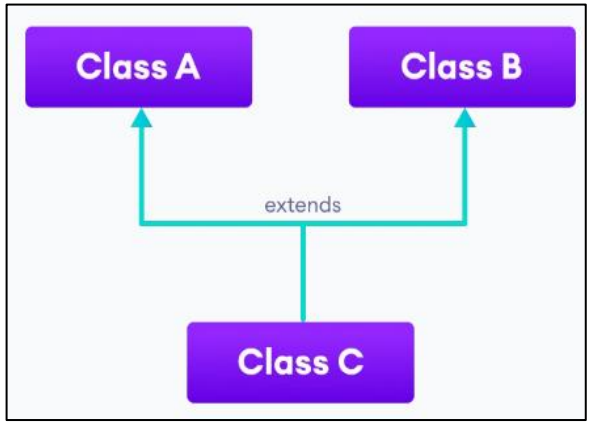
- There are 5 types of inheritances in general:
  1. **Single inheritance**
  2. **Multiple inheritance**
  3. **Multi-level inheritance**
  4. **Hierarchical inheritance**
  5. **Hybrid inheritance**

REMARK: Java does not support multiple inheritance.

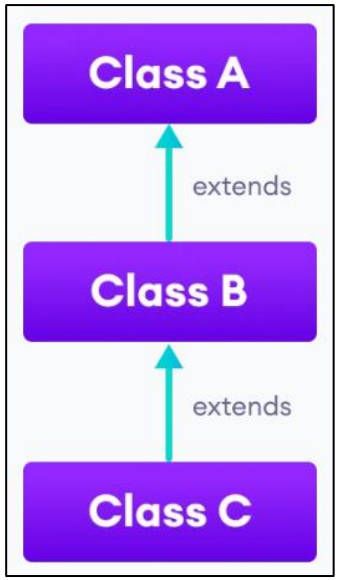
But it can achieve using **Interface** instead because we can implement multiple interface.



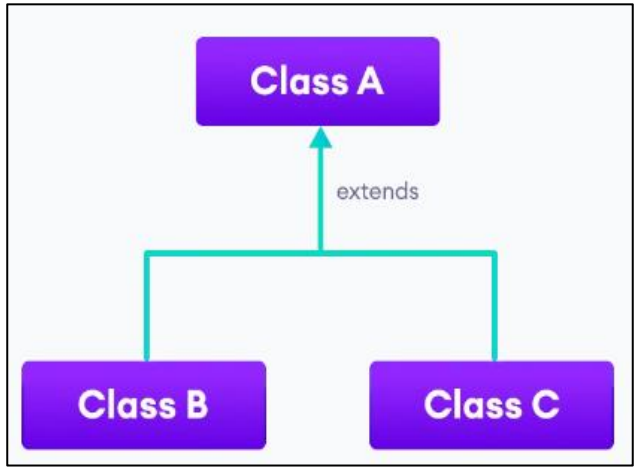
1) Single inheritance



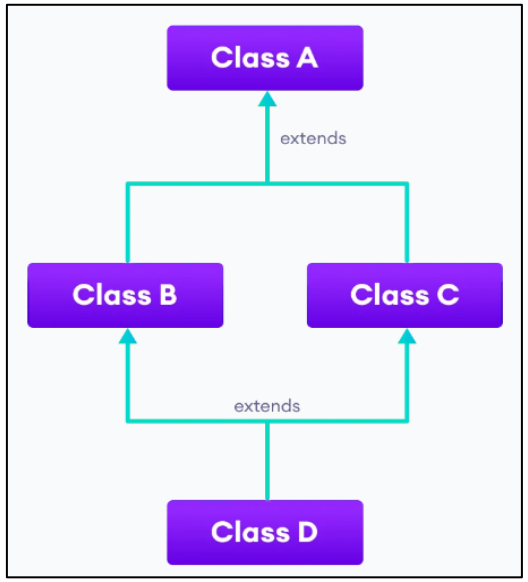
2) Multiple inheritance



3) Multi-level inheritance



4) Hierarchical inheritance



5) Hybrid inheritance

Q&A

Using special keywords

- Keyword **super**
- Keyword **final**

Keyword **super**

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }
}

class Child extends Parent {
    Child() {
        super(); // Calls Parent class constructor
        System.out.println("Child Constructor");
    }
}

public class SuperExample {
    public static void main(String[] args) {
        Child object = new Child();
    }
}
```

The super keyword is used to call the following from child class:

- Parent class methods
- Parent class constructors (can call in constructor of child class)
- Parent class variables

super keyword helps in accessing the parent class's methods and constructors from child class

Keyword **final**

Final Keyword in Inheritance:

- **final class**: Cannot be inherited.
- **final method**: Cannot be overridden.
- **final variable:** Cannot be changed.

Final Variable → To Create constant variable

Final Methods → Prevent Method Overriding

Final Classes → Prevent Inheritance

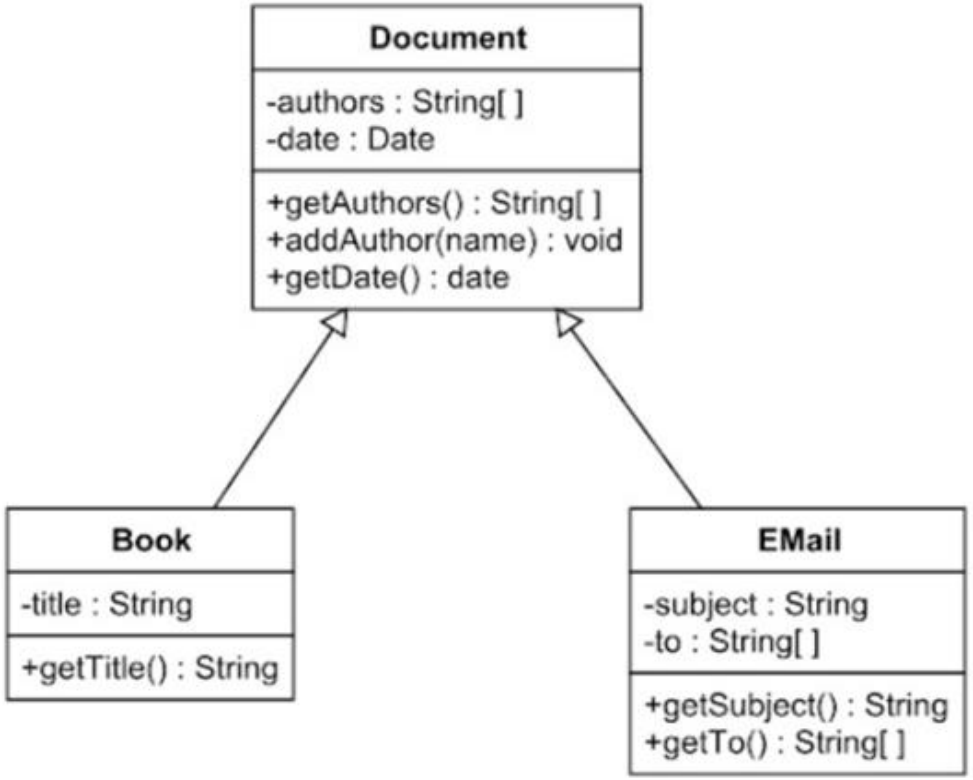
```
class A {
    final void display() {
        System.out.println("This is a final method");
    }
}

class B extends A {
    //Error. Can not override final method
    void display() {
        System.out.println("Trying to override");
    }
}

public class FinalExample {
    public static void main(String[] args) {
        B object = new B();
        object.display();
    }
}
```

Q&A

PRACTICE EXERCISES

Write a Java program to ……..

Create 3 classes representing the following class diagrams.



```java
class A {
    void show() {
        System.out.println("This is the Parent class");
    }
}

class B extends A {
    void display() {
        System.out.println("This is the Child class");
    }
}

public class Main {
    public static void main(String[] args) {
        B object = new B();
        object.show();          // Calling parent class method
        object.display();       // Calling child class method
    }
}
```

Single inheritance

```java
class Grandparent {
    void grandparentMethod() {
        System.out.println("This is Grandparent");
    }
}

class Parent extends Grandparent {
    void parentMethod() {
        System.out.println("This is Parent");
    }
}

class Child extends Parent {
    void childMethod() {
        System.out.println("This is Child");
    }
}

public class MultilevelExample {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.grandparentMethod(); // Grandparent's method
        obj.parentMethod();      // Parent's method
        obj.childMethod();       // Child's method
    }
}
```

Multi-level inheritance

```java
class Vehicle {
    void run() {
        System.out.println("Vehicle is running");
    }
}

class Car extends Vehicle {
    void fourWheels() {
        System.out.println("Car has four wheels");
    }
}

class Bike extends Vehicle {
    void twoWheels() {
        System.out.println("Bike has two wheels");
    }
}

public class HierarchicalExample {
    public static void main(String[] args) {
        Car car = new Car();
        car.run();
        car.fourWheels();

        Bike bike = new Bike();
        bike.run();
        bike.twoWheels();
    }
}
```

Hierarchical inheritance

Q&A